

Requirements Before YAML

How an architect turns a client's wishes into numbers and decisions:
features, order lifecycle, load math, availability, RPO/RTO, data protection and access

THE CORE IDEA

Beginners start with YAML. Architects start with **questions**. Every Kubernetes object we use later should exist because a **requirement** needs it. In an interview, "why did you use X?" should always have an answer that starts with a requirement.

Contents

1. From requirement to Kubernetes object
2. Functional requirements: roles, features, order lifecycle
3. Architect's pushback: the decisions and why
4. Non-functional 1: load (requests per second, step by step)
5. Non-functional 2: availability (the nines, in minutes)
6. Non-functional 3: RPO, RTO and the business case
7. Security 1: protecting data
8. Security 2: who may access the cluster
9. What's still open
10. Quiz and answer key

Decisions recorded in docs/00-project-charter.md §6 · 2026-09-25 · companion: docs/guides/high-availability-99.99.pdf

1. From requirement to Kubernetes object

The client says...	So we need...	Kubernetes object
"Admins upload menu photos"	Storage that several pods share	PV / PVC with ReadWriteMany (NFS)
"Daily sales report overnight"	A job on a schedule	CronJob
"Checkout must not fail when SMS is down"	An asynchronous queue	Redis Streams + a worker Deployment with no Service
"Nobody outside may call the discount service"	Network isolation	ClusterIP, no route + NetworkPolicy
"Updates without downtime"	Gradual replacement of pods	Deployment rolling update, readiness probes, PDB

2. Functional requirements

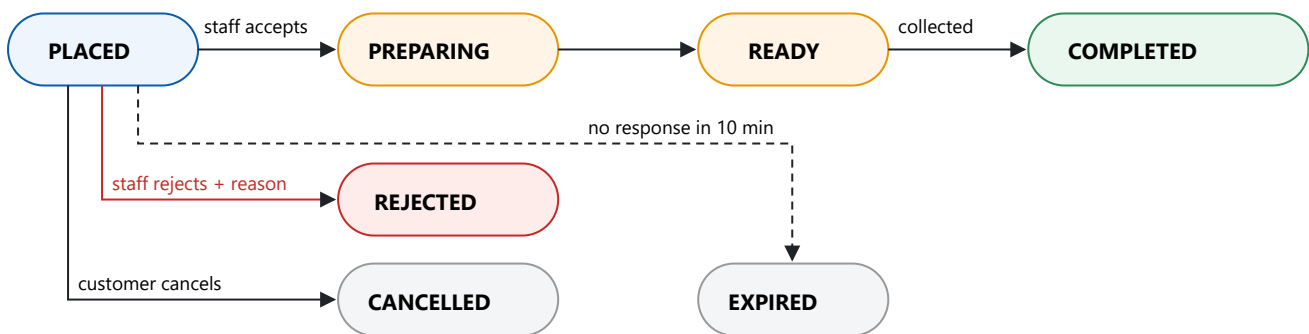
2.1 Roles

Role	Who	Can do
Customer	Anyone who signs up	Browse, order, cancel, see own orders
Staff	Counter staff	Accept/reject orders, move them through the lifecycle
Admin	Shop owner	Everything staff can do + menu, availability, coupons, reports

2.2 Features

#	Feature	Who	Leads to
F1	Browse the menu without logging in	Customer	menu-service (public), Redis cache
F2	Sign up / log in, opt-in checkbox for offers (off by default)	Customer	auth-service, JWT, Secret
F3	Place an order: login required, pickup only	Customer	order-service → discount-service (sync)
F4	Apply a coupon	Customer	discount-service, internal only
F5	Order history and live status	Customer	Postgres StatefulSet + PVC
F6	"Accepted / rejected / ready" message (fake SMS in logs)	Customer	notification worker via Redis Streams
F7	Add/edit menu items, upload photos	Admin	PVC ReadWriteMany
F8	Create coupons	Admin	discount-service
F9	Daily sales report	Admin	CronJob
F10	Expire old coupons nightly	System	CronJob
F11	Shop open/closed, items in/out of stock; unavailable items can't be ordered	Admin	menu-service; order-service validates
F12	Accept or reject orders (reason + suggestion text)	Staff	order-service
F13	Orders with no response in 10 min become EXPIRED	System	CronJob

2.3 The order lifecycle



Only a **PLACED** order can be cancelled by the customer, rejected, or expired. Rejection reasons: **CLOSED**, **OUT_OF_STOCK**, **OTHER**, plus a suggestion text ("no masala chai today, try ginger chai!"). Customers pay at the counter on pickup.

3. Architect's pushback: the decisions and why

Client wanted	Architect's response	Decision
Reject orders when closed or out of stock	Rejecting an order the customer already placed is a bad experience. Prevent it instead: show open/closed and stock on the menu	F11 added; REJECTED kept for surprises (the last milk ran out)
Offer a substitute ("we don't have X, want Y?")	A two-way negotiation needs a waiting state, notifications and timeouts, and teaches no new Kubernetes object	Rejection with a suggestion text; full substitution is out of scope
Login required (for contact details)	Agreed, and marketing messages need consent (India's DPDP Act 2023, GDPR)	Opt-in checkbox, off by default
Delivery?	Adds addresses (more personal data), a delivery step, tracking	Pickup only for now
(Unasked) What if staff never respond?	An order must not wait forever	EXPIRED after 10 min (F13)

INTERVIEW LINE

"I don't just take requirements; I challenge them. Where a rejection can be prevented, I prevent it. Where a feature adds complexity without matching value, I propose a simpler version. And every requirement is traced to the component that implements it."

4. Non-functional 1: load, step by step

Piece 1: a request

Every time a browser asks the server for something, that's 1 request, like one customer walking up to the counter and asking something.

Piece 2: "50 users online" ≠ "50 requests now"

50 customers inside a chai shop aren't all talking to the counter at the same second. Most are reading, drinking, chatting. Online is the same: **concurrent** means the site is *open*, not that everyone is clicking.

Piece 3: one user

1 user does something about once every 10 seconds
→ $1 \div 10 = 0.1$ requests per second

Piece 4: fifty users

$50 \text{ users} \times 0.1 = 5$ requests per second (average)

Piece 5: rush hour

5×4 (safety factor for the 8–10 am and 5–7 pm rush) = 20 requests per second ← design target

Piece 6: is 20 a lot?

No. One small server answers **thousands** of simple requests per second. That's why the charter honestly says this app doesn't *need* Kubernetes. We use it to learn. In the Phase 10 load test we'll push far beyond 20 on purpose to watch HPA scale.

WHERE DOES MOST OF THE TRAFFIC COME FROM?

Not browsing: **waiting**. A customer watching the status page checks every 5 s for ~5 minutes, which is ~60 requests. Knowing that, you could check less often or push updates to the browser instead.

Formula (Little's law style): requests/s $\approx$ users $\div$ seconds between clicks. 200 users $\rightarrow$ 20 req/s average $\rightarrow$ 80 at rush hour.

5. Non-functional 2: availability

Piece 1: turn the percentage into minutes

A month $\approx 30 \text{ days} \times 24 \text{ h} = 720 \text{ hours}$

99.5 % $\rightarrow$ down 0.5 % $\rightarrow 720 \times 0.005 = 3.6 \text{ hours per month} \leftarrow$ our target

99.9 % $\rightarrow$ down 0.1 % $\rightarrow 720 \times 0.001 = 0.72 \text{ h} \approx 43 \text{ minutes}$

99.99 % $\rightarrow$ down 0.01 % $\rightarrow 720 \times 0.0001 = 0.072 \text{ h} \approx 4.3 \text{ minutes}$

Piece 2: why not promise more?

We have **one** control plane, **one** Postgres and the NFS server on cka-m. If cka-m is down, running pods keep serving, but nothing can be changed, and anything using NFS stops. Promising 99.9% would be a lie; 99.5% is honest, and we'll measure it.

Piece 3: zero-downtime deploys are separate, and free

Planned updates must not take the site down. Kubernetes gives us this for free: rolling updates, readiness probes, PodDisruptionBudgets, at least 2 replicas of stateless services.

6. Non-functional 3: RPO, RTO and the business case

Term	Plain meaning	GaramChai
RPO	How much data you may lose : the time since the last good copy (look back)	4 hours $\rightarrow$ Postgres backup CronJob every 4 h
RTO	How long recovery may take (look forward)	1 hour

The money math, piece by piece

1 hour down at the evening rush: $50 \text{ orders} \times ₹100 = ₹5,000 \text{ lost}$

99.5 % allows 3.6 h/month; worst case all at rush hour: $3.6 \times ₹5,000 = ₹18,000/\text{month}$

99.99 % would need multi-site servers, a replicated database and a 24x7 on-call team:

one engineer's salary alone is more than ₹18,000/month, before a single server

$\rightarrow$ spending lakhs to save ₹18,000 is like a ₹10 lakh alarm for a ₹20,000 bicycle

Recommendation	Why	Cost
99.5 % (count the hours that matter)	Downtime at 3 am costs ₹0	nothing extra
Zero-downtime deploys	Updates shouldn't cause outages	free (built in)
A tested backup	Lost data is worse than downtime	a CronJob + a drill
Revisit when the business grows	20 shops $\rightarrow$ 1 hour down could cost ₹1 lakh	nothing now

THE LESSON

Availability is a **business** decision. Put two numbers side by side: what downtime **costs**, and what preventing it **costs**. Pick the level where the second is smaller. The full 99.99% treatment (SLA/SLO/SLI, MTTR, multi-region AWS, cost) is in [docs/guides/high-availability-99.99.pdf](#).

#	Requirement	Target
N1	Load	50 concurrent users; ~5 req/s average, ~20 at peak
N2	Availability	99.5 %; zero downtime during deploys
N3	Speed	p95 < 300 ms (95 of 100 requests faster than this)
N4	Orders data	RPO 4 h, RTO 1 h, backups kept 7 days
N5	Retention	Order history kept 1 year, then deleted

7. Security 1: protecting data

7.1 List the data and rank it

Data	If it leaks...
Menu, prices	Nothing, it's public
Orders	A little embarrassing
Email + phone	Personal data: spam, scams, legal trouble
Passwords	Very bad: people reuse them for bank and email
Keys, DB passwords	Worst: an attacker becomes the app

7.2 Passwords: store a fingerprint, not the password

```
Sign-up: "chai@123" → bcrypt → "$2b$12$Kx9...f3" ← only this is saved
Login:   "chai@123" → bcrypt → same fingerprint? → yes = correct password
```

- A hash is **one-way**: a stolen hash can't be turned back into the password.
- But attackers can still **guess**: hash a million common passwords and compare. **bcrypt** defends by being **slow on purpose** and adding a random **salt** per user (same password → different hash).

7.3 A Secret isn't secret by default

- A Kubernetes Secret is only **base64-encoded**: `echo cGFzc3dvcmQ= | base64 -d` gives password to anyone.
- It's readable by anyone with RBAC `get secrets`, and by anyone with access to **etcd or its backups**.
- **Encryption at rest** (Phase 8) encrypts Secrets inside etcd with a **symmetric** key (e.g. AES): one key both encrypts and decrypts. Private/public key pairs (asymmetric) are a different thing, used in TLS certificates.

#	Rule	Leads to
S1	Passwords only as bcrypt hashes	auth-service code
S2	Keys and DB passwords only in Secrets; only <code>*.example.yaml</code> in Git	Secret, <code>.gitignore</code>
S3	Encryption at rest for Secrets in etcd	EncryptionConfiguration (Phase 8)
S4	One DB user per service; only auth-service reads personal data	Postgres roles (Phase 5)
S5	TLS at the edge (browser → Traefik); inside, default-deny NetworkPolicy	Ingress/Gateway + cert-manager, NetworkPolicy

Not chosen: mTLS between services everywhere. It needs a service mesh, which is too heavy for our RAM (common in banks and large shared clusters).

8. Security 2: who may access the cluster

Step	Question	Failure
Authentication	Who are you?	401
Authorization (RBAC)	What may you do?	403

Humans are names in certificates (`CN` = name, `O` = group) signed by the cluster CA; there's no User object.

Programs are ServiceAccount objects. Client certificates **can't be revoked**, so a stolen admin certificate works until it expires; that's why the CA key and admin.conf are guarded (details in study notes 1, §11).

#	Who	Gets
A1	Vishal (admin)	admin.conf: never shared, never committed
A2	"intern" user	Read-only in <code>garamchai</code> only (CSR + Role/RoleBinding, Phase 8)
A3	Each service	Own ServiceAccount, no API permissions, <code>automountServiceAccountToken: false</code>
A4	Backup Job	Small Role in <code>garamchai-data</code> only
A5	SSH to nodes	Vishal only, key-based

Deployments: CI (GitHub Actions) only builds and pushes images; deploys run from the laptop. The cluster sits behind VMware NAT, so GitHub can't reach it, and opening it would put cluster credentials on a third-party server. The real-world alternative is **GitOps**: an agent inside the cluster (e.g. Argo CD) *pulls* from Git, so no credentials leave the cluster.

9. What's still open

- **Operations** section of the questionnaire: releases and rollback, logging, monitoring.
- Then: architecture document v1.0 and ADRs for new decisions → Phase 1 done.
- Before Phase 3: automatic pull of etcd backups to Windows (R10c-4).

10. Quiz

1. Why do architects start with requirements instead of YAML? Give one requirement → object example.
2. Which three roles does GaramChai have, and what can staff do that customers can't?
3. From which state can an order be cancelled by the customer? What happens if staff never respond?
4. Why did we add F11 (open/closed, stock) instead of just rejecting orders?
5. Why is item substitution out of scope?
6. Why does the sign-up form need an opt-in checkbox?
7. 50 users each click once every 10 s. How many requests per second on average, and at rush hour (×4)?
8. 200 users, same behaviour: average and rush-hour requests per second?
9. Which user activity causes the most requests, and why?
10. How many hours per month of downtime does 99.5 % allow? Show the math.
11. Why is 99.9 % not an honest promise for our lab?
12. Define RPO and RTO in one line each. What's our RPO for orders, and what does it force us to build?
13. The site is down for 2 hours at rush hour. How much is lost (₹5,000/hour)?
14. Why is 99.99 % a bad deal for GaramChai? Use numbers.
15. Why store a bcrypt hash instead of the password? What do slowness and salt add?
16. Why is a Kubernetes Secret not truly secret by default? Who can read it?
17. Does encryption at rest use a private key? Explain.
18. Why TLS at the edge and not mTLS everywhere for us?
19. Why do our services get no ServiceAccount token?
20. Why don't we let GitHub Actions deploy to the cluster? What's the GitOps alternative?

Answer key

1. Every object should exist because a requirement needs it; that's what makes design defensible. E.g. "admins upload photos" → ReadWriteMany PVC; "nightly report" → CronJob.
2. Customer, staff, admin. Staff accept/reject orders and move them through PREPARING → READY → COMPLETED.
3. Only while PLACED. With no staff response within 10 min it becomes EXPIRED.
4. Prevention beats rejection: customers shouldn't be able to place orders that will be rejected. REJECTED stays for surprises.
5. It needs a waiting state, notifications and timeouts (a negotiation workflow) and teaches no new Kubernetes object. A suggestion in the rejection message covers most of the value.
6. Sending offers requires consent under India's DPDP Act 2023 (and GDPR); it's off by default.
7. $50 \times 0.1 = 5$ req/s average; $\times 4 = 20$ req/s at rush hour.
8. $200 \times 0.1 = 20$ req/s average; 80 at rush hour.
9. Waiting for READY: the status page checks every 5 s for about 5 minutes (~60 requests per order).
10. $720 \text{ h} \times 0.005 = 3.6$ hours.
11. Single control plane, single Postgres, NFS on cka-m: one machine failing takes parts of the app down for longer than 43 minutes.
12. RPO: how much data you may lose (time since the last good copy). RTO: how long recovery may take. RPO 4 h → a Postgres backup CronJob every 4 hours (plus a restore drill).
13. $2 \times ₹5,000 = ₹10,000$.
14. 99.5 % worst case loses ~₹18,000/month; 99.99 % needs multi-site infrastructure and a 24×7 team costing lakhs per month. You'd spend far more than the downtime costs.
15. A hash is one-way, so stolen hashes don't reveal passwords. Slowness makes mass guessing take years; a per-user salt makes identical passwords hash differently.
16. It's only base64-encoded. Anyone with RBAC `get secrets`, or with access to etcd or its backups, can read it.
17. No. It uses a symmetric key (e.g. AES) that both encrypts and decrypts. Private/public keys are asymmetric crypto, used for TLS.
18. mTLS everywhere needs a service mesh, too heavy for our RAM. TLS at the edge + default-deny NetworkPolicy is proportionate.
19. Our apps never call the Kubernetes API. Without a token, a hacked pod gives the attacker no cluster access (least privilege).
20. GitHub can't reach a cluster behind NAT, and exposing it would put cluster credentials on a third-party server. GitOps: an in-cluster agent like Argo CD pulls changes from Git.